LAPORAN UJIAN TENGAH SEMESTER

Pemrograman / Coding Akuntansi

Fakultas Ekonomika dan Bisnis — Universitas Diponegoro

Nama : Arfa Idraki Rori
NIM : 12030124130110
Kelas : S1 Akuntansi Reguler
Mata Kuliah : Pemrograman / Coding
Dosen Pengampu : Totok Dewayanto
Tanggal Ujian : Senin, 19 Oktober 2026
Waktu : 120 Menit
Sifat Ujian : Open Book / Open AI / Open Code Sandbox

🔗 **Link Repositori GitHub (Public):**
https://github.com/arfarori23/UTS-Coding-Akuntansi-Arfa-12030124130110

DAFTAR ISI

1. Soal 1 — Desain Arsitektur Data, Normalisasi & Asas Berpasangan
2. Soal 2 — Otomasi Bisnis, Stored Procedures & Trigger Persediaan
3. Soal 3 — Integrasi Purwarupa Web Full-Stack & Orkestrasi AI Tools
4. Kesimpulan
5. Lampiran

SOAL 1 — DESAIN ARSITEKTUR DATA, NORMALISASI & ASAS BERPASANGAN

Bobot: 25% | CPMK-1 / CPL-3, CPL-4

Konteks Masalah

Sebuah entitas ritel modern sedang mengembangkan modul General Ledger mandiri. Sistem akuntansi terdahulu mengalami anomali saldo tidak seimbang (*unbalanced balance sheet*) akibat pencatatan jurnal hanya mengandalkan pengecekan di sisi antarmuka pengguna (*frontend validation*), sehingga transaksi yang dikirim langsung melalui API dapat lolos tanpa pasangan Debit-Kredit yang valid.

1.1 Perancangan Skema Relasional (ERD)

Skema basis data dirancang dengan **3 entitas utama** menggunakan model **Header–Line**:

a. Entitas dan Atributnya

Entitas	Primary Key	Atribut Utama
chart_of_accounts	account_id	account_code, account_name, normal_balance, report_class, created_at
journal_headers	header_id	trx_date, reference_no, description, status_posting, created_by, created_at
journal_lines	line_id	header_id (FK), account_id (FK), debit, credit

b. Relasi Antar Entitas

```
chart_of_accounts (1) ──────< (N) journal_lines
                                        │
                                        │ (N)
                                        │
                                        ▼
                                journal_headers (1)
```

Kardinalitas:

- journal_headers (1) ──< (N) journal_lines → satu voucher jurnal punya banyak baris debit/kredit
- chart_of_accounts (1) ──< (N) journal_lines → satu akun muncul di banyak baris jurnal

c. Diagram ERD


ERD

Gambar ERD lengkap tersedia di file soal-1-erd-ddl/erd.png

1.2 Kekangan Integritas (DDL PostgreSQL)

File lengkap: soal-1-erd-ddl/schema.sql

a. Tabel 1: chart_of_accounts

```sql
CREATE TABLE chart_of_accounts (
    account_id      SERIAL       PRIMARY KEY,
    account_code    VARCHAR(20)  NOT NULL UNIQUE,
    account_name    VARCHAR(100) NOT NULL,
    normal_balance  VARCHAR(6)   NOT NULL
                    CHECK (normal_balance IN ('DEBIT','CREDIT')),
    report_class    VARCHAR(20)  NOT NULL
                    CHECK (report_class IN
('ASSET','LIABILITY','EQUITY','REVENUE','EXPENSE')),
    created_at      TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

b. Tabel 2: journal_headers

```
CREATE TABLE journal_headers (
    header_id       SERIAL       PRIMARY KEY,
    trx_date        DATE         NOT NULL,
    reference_no    VARCHAR(50)  NOT NULL UNIQUE,
    description     TEXT,
    status_posting  VARCHAR(10)  NOT NULL DEFAULT 'DRAFT'
                    CHECK (status_posting IN ('DRAFT','POSTED','REVERSED')),
    created_by      VARCHAR(50)  NOT NULL DEFAULT current_user,
    created_at      TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

c. Tabel 3: journal_lines

```
CREATE TABLE journal_lines (
    line_id     SERIAL        PRIMARY KEY,
    header_id   INT           NOT NULL,
    account_id  INT           NOT NULL,
    debit       NUMERIC(15,2) NOT NULL DEFAULT 0,
    credit      NUMERIC(15,2) NOT NULL DEFAULT 0,

    CONSTRAINT fk_lines_header
        FOREIGN KEY (header_id) REFERENCES journal_headers(header_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_lines_account
        FOREIGN KEY (account_id) REFERENCES chart_of_accounts(account_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,

    CONSTRAINT chk_debit_non_negative  CHECK (debit  >= 0),
    CONSTRAINT chk_credit_non_negative CHECK (credit >= 0),
    CONSTRAINT chk_not_both_zero       CHECK (NOT (debit = 0 AND credit = 0)),
    CONSTRAINT chk_not_both_filled     CHECK (NOT (debit > 0 AND credit > 0))
);
```

d. Ringkasan Constraint yang Diterapkan

Constraint	Tujuan
`PRIMARY KEY`	Identitas unik tiap baris
`FOREIGN KEY ON DELETE RESTRICT`	Cegah orphan journal & referensi hilang
`CHECK (debit >= 0)` & `CHECK (credit >= 0)`	Cegah nilai negatif
`CHECK NOT (debit = 0 AND credit = 0)`	Cegah baris kosong
`CHECK NOT (debit > 0 AND credit > 0)`	Paksa satu baris hanya debit ATAU kredit

e. Hasil Uji Constraint

8 test case dijalankan (lihat `constraint_tests.sql`):

- ✅ Test 1: insert valid → sukses
- ❌ Test 2: normal_balance invalid → ERROR (sesuai harapan)
- ❌ Test 3: debit negatif → ERROR
- ❌ Test 4: debit & kredit nol → ERROR
- ❌ Test 5: FK header_id tidak ada → ERROR
- ❌ Test 6: FK account_id tidak ada → ERROR
- ❌ Test 7: DELETE header yang masih dipakai → ERROR
- ✅ Test 8: insert valid → sukses

Screenshot bukti di `docs/screenshots/soal-1/`.

1.3 Analisis Audit & Keamanan Finansial

a. Pemisahan Header–Line Lebih Andal dari Tabel Tunggal

1) Sudut Pandang Jejak Audit (Audit Trail)

1. **Kontrol status terpusat.** Kolom `status_posting` hanya ada di header → penguncian seluruh voucher cukup 1 operasi UPDATE.
2. **Nomor bukti unik di satu tempat.** `reference_no` tidak berulang (di tabel tunggal, akan duplikat di setiap baris).
3. **Immutability selektif.** Trigger kunci `POSTED` di header + FK `RESTRICT` di line = detail otomatis terkunci tanpa harus mengunci baris satu-satu.
4. **Reversing entry lebih bersih.** Storno = header baru yang merefer ke header asal, tanpa menandai tiap baris detail.

2) Sudut Pandang Bentuk Normal Ketiga (3NF)

Aturan 3NF: setiap atribut non-kunci hanya boleh bergantung pada primary key-nya (tidak ada transitive dependency).

Aspek	Tabel Tunggal	Header–Line (3NF)
`trx_date`, `reference_no`, `description`	Berulang tiap baris → langgar 2NF	Hanya di header → 3NF ✅
`account_name`, `normal_balance`	Berulang tiap baris	Ada di `chart_of_accounts` → 3NF ✅
Update nama akun	Harus update N baris	Cukup 1 baris di COA
Risiko anomali update	Tinggi	Nol

Anomali yang dihindari:

- **Update anomaly:** ganti nama akun → kalau ada 10.000 baris, harus update semua.

- **Insert anomaly:** tidak bisa tambah akun baru tanpa transaksi.
- **Delete anomaly:** hapus baris terakhir suatu akun → kehilangan info akun.

b. Rekapitulasi Keputusan Desain

Kriteria	Tabel Tunggal	Header–Line
Memenuhi 3NF	❌	✅
Audit trail granular	❌	✅
Immutability mudah	❌	✅
Storage	Lebih kecil	Sedikit lebih besar

Kesimpulan: Meskipun header–line memerlukan JOIN, manfaat dari sisi integritas, audit, dan normalisasi jauh melebihi trade-off performa yang bisa diatasi dengan indexing (`idx_lines_header`, `idx_lines_account`).

SOAL 2 — OTOMASI BISNIS, STORED PROCEDURES & TRIGGER PERSEDIAAN

Bobot: 40% | CPMK-2 / CPL-3

Konteks Masalah

PT Diponegoro Logistik Utama menggunakan metode penilaian persediaan berbasis sistem **perpetual**. Masalah sering terjadi saat staf operasional mencatat retur atau pengeluaran barang: saldo fisik di gudang berubah, namun nilai nominal akun **Persediaan Barang Dagang** dan **Beban Pokok Pendapatan (HPP)** pada buku besar terlambat dimutakhirkan, sehingga memicu selisih saat stock opname.

2.1 Otomatisasi Mutasi via Database Trigger

File: `soal-2-trigger/02_trigger_function.sql`

a. Struktur Tabel Dasar

```
CREATE TABLE inventory_items (
    item_id    VARCHAR(10) PRIMARY KEY,
    item_name  VARCHAR(100) NOT NULL,
    stock_qty  INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),
    unit_cost  NUMERIC(15, 2) NOT NULL CHECK (unit_cost >= 0)
);

CREATE TABLE inventory_transactions (
    trx_id    SERIAL PRIMARY KEY,
    item_id   VARCHAR(10) NOT NULL REFERENCES inventory_items(item_id),
    trx_type  VARCHAR(10) NOT NULL CHECK (trx_type IN
('PURCHASE','SALE','ADJUSTMENT')),
```

```
    qty       INT NOT NULL,
    trx_date  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

b. Trigger Function

```
CREATE OR REPLACE FUNCTION trg_update_stock_and_journal()
RETURNS TRIGGER AS $$
DECLARE
    v_current_stock INT;
BEGIN
    -- Kunci baris item untuk cegah race condition
    SELECT stock_qty INTO v_current_stock
    FROM inventory_items
    WHERE item_id = NEW.item_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Item dengan ID % tidak ditemukan', NEW.item_id;
    END IF;

    IF NEW.trx_type = 'PURCHASE' THEN
        IF NEW.qty <= 0 THEN
            RAISE EXCEPTION 'PURCHASE harus qty > 0';
        END IF;
        UPDATE inventory_items
        SET stock_qty = stock_qty + NEW.qty
        WHERE item_id = NEW.item_id;

    ELSIF NEW.trx_type = 'SALE' THEN
        IF NEW.qty <= 0 THEN
            RAISE EXCEPTION 'SALE harus qty > 0';
        END IF;

        IF v_current_stock - NEW.qty < 0 THEN
            RAISE EXCEPTION 'Stok tidak cukup untuk item %. Stok tersedia: %,
diminta: %',
                  NEW.item_id, v_current_stock, NEW.qty;
        END IF;

        UPDATE inventory_items
        SET stock_qty = stock_qty - NEW.qty
        WHERE item_id = NEW.item_id;

    ELSIF NEW.trx_type = 'ADJUSTMENT' THEN
        IF v_current_stock + NEW.qty < 0 THEN
            RAISE EXCEPTION 'Adjustment menyebabkan stok minus';
        END IF;
        UPDATE inventory_items
        SET stock_qty = stock_qty + NEW.qty
        WHERE item_id = NEW.item_id;
```

```
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_inventory_update
AFTER INSERT ON inventory_transactions
FOR EACH ROW
EXECUTE FUNCTION trg_update_stock_and_journal();
```

c. Hasil Uji Coba

7 test case (lihat `04_test_cases.sql`):

#	Test	Hasil
1	PURCHASE 10 unit	✅ Stok naik jadi 10
2	SALE 3 unit	✅ Stok turun jadi 7
3	SALE 100 unit (melebihi stok)	❌ ERROR "Stok tidak cukup"
4	ADJUSTMENT -2	✅ Stok jadi 5
5	ADJUSTMENT -100 (minus)	❌ ERROR "Adjustment menyebabkan stok minus"
6	Item tidak ada	❌ ERROR foreign key
7	Cek stok akhir	✅ Konsisten

Screenshot bukti di `docs/screenshots/soal-2/`.

2.2 Penanganan Concurrency (Race Condition)

a. Skenario Race Condition

```
Waktu │ Kasir A                          │ Kasir B
──────┼──────────────────────────────────┼──────────────────────────────
t=0   │ Baca stok = 1                    │
t=1   │                                  │ Baca stok = 1
t=2   │ Cek: 1 - 1 = 0 → OK              │
t=3   │                                  │ Cek: 1 - 1 = 0 → OK
t=4   │ Jual 1 → stok jadi 0             │
t=5   │                                  │ Jual 1 → stok jadi -1 (OVERSELING!)
t=6   │ COMMIT                           │ COMMIT
```

Ini disebut **lost update** — dua transaksi baca nilai lama, lalu dua-duanya menulis nilai baru berdasarkan nilai lama tersebut.

b. Solusi 1 — Pessimistic Locking (`SELECT ... FOR UPDATE`)

Cara kerja:

1. Saat kasir A melakukan `SELECT ... FOR UPDATE`, baris item **dikunci**.
2. Kasir B yang mencoba lock baris yang sama akan **menunggu** (block) sampai kasir A commit/rollback.
3. Setelah kasir A commit, kasir B dapat lock & baca stok terbaru (0).
4. Trigger kasir B mendeteksi stok 0 < 1 → `RAISE EXCEPTION` → rollback.

```
BEGIN;
    SELECT stock_qty FROM inventory_items
    WHERE item_id = 'BRG001'
    FOR UPDATE;

    INSERT INTO inventory_transactions (item_id, trx_type, qty)
    VALUES ('BRG001', 'SALE', 1);
COMMIT;
```

Keunggulan:

- Tidak perlu retry logic (aplikasi tidak perlu tangani exception khusus).
- Deterministic — pasti salah satu menang, satunya gagal.

Kekurangan:

- Menurunkan throughput (kasir B harus menunggu).
- Risiko **deadlock** kalau ada 2+ barang yang di-lock dengan urutan berbeda.

c. Solusi 2 — Serializable Isolation Level

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
    INSERT INTO inventory_transactions (item_id, trx_type, qty)
    VALUES ('BRG001', 'SALE', 1);
COMMIT;
```

Cara kerja:

- PostgreSQL otomatis deteksi konflik dan rollback salah satu transaksi.
- Aplikasi **wajib retry** kalau dapat error `could not serialize access due to concurrent update`.

Keunggulan: tidak perlu pikir locking manual; cocok untuk transaksi kompleks. **Kekurangan:** aplikasi harus punya retry loop; overhead deteksi konflik lebih besar.

d. Rekomendasi untuk PT Diponegoro Logistik Utama

Pilih **Pessimistic Locking (`SELECT ... FOR UPDATE`)** karena:

1. Update-nya single-row (satu item per transaksi).
2. Deterministic lebih penting daripada throughput.

3. Sudah otomatis diimplementasikan di dalam fungsi trigger (baris `SELECT ... FOR UPDATE` di `02_trigger_function.sql`).

SOAL 3 — INTEGRASI PURWARUPA WEB FULL-STACK & ORKESTRASI AI TOOLS

Bobot: 35% | CPMK-3 / CPL-3, CPL-4

Konteks Masalah

Auditor internal meminta pembuatan purwarupa modul **Voucher Entri Transaksi Kas Keluar** berbasis web yang terhubung dengan basis data cloud (Supabase/PostgreSQL). Antarmuka harus memastikan asas **zero-discrepancy** (Total Debit = Total Kredit) sebelum tombol Submit aktif, serta dilengkapi pencatatan riwayat audit (*audit log*).

3.1 Orkestrasi AI Tools

a. Gemini Canvas — Generate UI Voucher

Tools: Gemini Canvas (gemini.google.com)
Stack: HTML + Tailwind CSS + JavaScript (vanilla)
Output: `soal-3-fullstack/frontend/index.html` + `app.js`
Dokumentasi prompt: `soal-3-fullstack/frontend/prompts/gemini_canvas_prompt.md`

Prompt yang Digunakan (Ringkasan)

```
Buatkan saya satu file HTML lengkap (termasuk Tailwind CSS via CDN dan
JavaScript vanilla) untuk formulir "Voucher Entri Kas Keluar" dengan
spesifikasi:
- Header form (tanggal, no. voucher, keterangan)
- Tabel dinamis debit/kredit dengan add/remove row
- Live balance indicator real-time
- Tombol Submit auto-disable kalau tidak balance
- Styling Tailwind CSS dengan font Inter
```

Fitur UI yang Dihasilkan

Fitur	Deskripsi
Header form	Tanggal, nomor voucher, keterangan
Tabel dinamis	Baris debit/kredit, tombol + dan ×
Live balance	Total Debit, Total Kredit, Selisih real-time
Badge status	✅ BALANCE / ❌ TIDAK BALANCE

Fitur	Deskripsi
Tombol Submit	Disabled sampai balance & field terisi

Modifikasi Manual

1. Pisahkan JavaScript ke file `app.js` terpisah.
2. Tambahkan koneksi Supabase via `@supabase/supabase-js` (CDN).
3. Fungsi `submitJournal()` untuk insert ke tabel `journal_headers` & `journal_lines`.

Kenapa HTML+Tailwind (bukan Streamlit):

- Tidak butuh Python environment → portabel, cukup buka `.html`.
- Bisa dideploy ke Vercel/Netlify dalam 1 klik.
- Lebih mendekati arsitektur produksi modern.

b. NotebookLM — Verifikasi PSAK 2

Tools: NotebookLM (notebooklm.google.com)
Sumber: PDF PSAK 2 + `index.html` + `app.js` (dikonversi ke `.md`)
Output: `soal-3-fullstack/psak_verification.md`

Pertanyaan yang Diajukan

1. Apakah logika voucher kas keluar di kode saya sudah sesuai kaidah pengakuan Kas dan Setara Kas? Berikan checklist kepatuhan.
2. Menurut PSAK 2, apa syarat instrumen diklasifikasikan sebagai setara kas? Apakah kode saya sudah memisahkan akun kas vs setara kas?
3. Apakah ada pelanggaran aturan bisnis PSAK 2 di kode saya? Kalau ada, tunjukkan di baris/function mana.

Ringkasan Hasil Verifikasi

Aspek PSAK 2	Hasil	Catatan
Pengakuan kas (paragraf 7-8)	✅ Sesuai	Asas berpasangan debit-kredit terpenuhi
Setara kas (paragraf 9)	⚠️ Perlu perbaikan	Perlu pemisahan eksplisit akun Kas vs Setara Kas di COA
Audit trail (PSAK 1)	✅ Sesuai	Metadata created_by, created_at, hash_checksum lengkap

Rekomendasi perbaikan:

1. Tambah kolom `jenis_kas` ('KAS'/'SETARA_KAS') di COA.
2. Tambah validasi di `app.js`: akun kas wajib dari COA bertipe KAS.
3. Tambah nomor urut voucher otomatis.

3.2 Mekanisme Jejak Audit (Audit Trail & Immutability)

File: `soal-3-fullstack/backend/audit_trail.sql` & `reversing_entry_rpc.sql`

a. Kolom Audit Tambahan

```
ALTER TABLE journal_headers
    ADD COLUMN created_by     VARCHAR(50) DEFAULT current_user,
    ADD COLUMN created_at     TIMESTAMP   DEFAULT CURRENT_TIMESTAMP,
    ADD COLUMN posted_by      VARCHAR(50),
    ADD COLUMN posted_at      TIMESTAMP,
    ADD COLUMN hash_checksum  TEXT,
    ADD COLUMN reverses_id    INT REFERENCES journal_headers(header_id);
```

b. Trigger Immutability

Jurnal berstatus `POSTED` tidak boleh diubah/dihapus:

```
CREATE OR REPLACE FUNCTION fn_lock_posted_journal()
RETURNS TRIGGER AS $$
BEGIN
    -- Bypass untuk operasi internal (reversing entry)
    IF current_setting('app.allow_posted_update', true) = 'on' THEN
        RETURN COALESCE(NEW, OLD);
    END IF;

    IF TG_OP = 'UPDATE' AND OLD.status_posting = 'POSTED' THEN
        RAISE EXCEPTION 'Jurnal POSTED tidak boleh diubah. Gunakan reversing
entry.';
    END IF;

    IF TG_OP = 'DELETE' AND OLD.status_posting = 'POSTED' THEN
        RAISE EXCEPTION 'Jurnal POSTED tidak boleh dihapus. Gunakan reversing
entry.';
    END IF;

    IF TG_OP = 'UPDATE'
       AND OLD.status_posting = 'DRAFT'
       AND NEW.status_posting = 'POSTED' THEN
        NEW.posted_by := current_user;
        NEW.posted_at := CURRENT_TIMESTAMP;
    END IF;

    RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_lock_posted_journal
BEFORE UPDATE OR DELETE ON journal_headers
```

```
FOR EACH ROW
EXECUTE FUNCTION fn_lock_posted_journal();
```

c. Hash Checksum SHA-256

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE OR REPLACE FUNCTION fn_generate_journal_hash(p_header_id INT)
RETURNS TEXT AS $$
DECLARE
    v_payload TEXT;
BEGIN
    SELECT string_agg(
        h.header_id || '|' || h.trx_date || '|' || h.reference_no || '|' ||
        COALESCE(h.description,'') || '|' ||
        l.line_id || '|' || l.account_id || '|' || l.debit || '|' || l.credit,
        E'\n' ORDER BY l.line_id
    )
    INTO v_payload
    FROM journal_headers h
    LEFT JOIN journal_lines l ON l.header_id = h.header_id
    WHERE h.header_id = p_header_id;

    RETURN encode(digest(v_payload, 'sha256'), 'hex');
END;
$$ LANGUAGE plpgsql;
```

Prinsip: `hash = SHA256(header_id | trx_date | ref_no | desc | line_id | account_id | debit | credit)`

Kalau ada yang mengubah 1 karakter saja → hash berubah → verifikasi ulang `verify_integrity()` akan return `False`.

d. Reversing Entry (Storno)

```
CREATE OR REPLACE FUNCTION rpc_create_reversing_entry(p_original_header_id INT)
RETURNS INT AS $$
DECLARE
    v_new_header_id INT;
    v_orig          RECORD;
BEGIN
    SELECT * INTO v_orig FROM journal_headers
    WHERE header_id = p_original_header_id;

    IF v_orig.status_posting <> 'POSTED' THEN
        RAISE EXCEPTION 'Hanya jurnal POSTED yang bisa dibalik';
    END IF;

    -- Bikin header baru (jurnal pembalik)
```

```
    INSERT INTO journal_headers (trx_date, reference_no, description,
status_posting, reverses_id)
    VALUES (CURRENT_DATE, 'REV-' || v_orig.reference_no,
            'Reversing entry untuk ' || v_orig.reference_no,
            'POSTED', p_original_header_id)
    RETURNING header_id INTO v_new_header_id;

    -- Copy baris dengan debit & kredit DITUKAR
    INSERT INTO journal_lines (header_id, account_id, debit, credit)
    SELECT v_new_header_id, account_id, credit, debit
    FROM journal_lines WHERE header_id = p_original_header_id;

    -- Bypass trigger untuk update status
    PERFORM set_config('app.allow_posted_update', 'on', true);
    UPDATE journal_headers SET status_posting = 'REVERSED'
    WHERE header_id = p_original_header_id;

    RETURN v_new_header_id;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

Alur reversing entry:

1. Jurnal asli POSTED (mis. header_id=1) salah.
2. Panggil `SELECT rpc_create_reversing_entry(1)`.
3. Sistem bikin header baru (header_id=2) dengan `reference_no = "REV-JV-2026-0001"`, `reverses_id=1`, dan **debit/kredit ditukar**.
4. Status jurnal asli → `REVERSED`.
5. Kedua jurnal tetap ada → jejak audit terjaga.

e. Hasil Uji Coba

#	Test	Hasil
H.3	UPDATE jurnal POSTED	❌ ERROR (sesuai harapan)
H.4	DELETE jurnal POSTED	❌ ERROR (sesuai harapan)
H.5	Reversing entry	✅ Berhasil buat header baru
H.6	UI balance indicator	✅ Tombol Submit dinamis

Screenshot bukti di `docs/screenshots/soal-3/`.

KESIMPULAN

Melalui UTS ini, tiga pilar utama sistem akuntansi terkomputerisasi berhasil diimplementasikan:

1. Integritas Data (Soal 1)

Skema relasional **3NF** dengan **constraint ketat** (PK, FK RESTRICT, CHECK) menjamin tidak ada jurnal tidak balance yang lolos. Pemisahan **header–line** memungkinkan audit trail granular dan immutability selektif.

2. Otomasi & Pengendalian (Soal 2)

Trigger database memastikan setiap mutasi persediaan langsung memutakhirkan saldo, dengan proteksi **anti-oversell** dan **anti-race-condition** via `SELECT ... FOR UPDATE`.

3. Kepatuhan & Audit (Soal 3)

Purwarupa web dengan **audit trail lengkap** (immutability, hash checksum, reversing entry) sesuai standar PSAK & praktik audit modern. Verifikasi PSAK 2 via **NotebookLM** menunjukkan logika aplikasi sudah compliant.

Prinsip Kunci

> **Validasi di level database (*defense in depth*) lebih andal daripada hanya mengandalkan validasi frontend.**

Frontend validation bisa di-bypass via API langsung. Constraint database, trigger, dan RLS adalah pertahanan terakhir yang tidak bisa dilewati.

LAMPIRAN

A. Struktur Repositori

```
UTS-Coding-Akuntansi-Arfa-12030124130110/
├── README.md
├── .gitignore
├── .env.example
├── soal-1-erd-ddl/
│   ├── schema.sql
│   ├── sample_data.sql
│   ├── constraint_tests.sql
│   ├── ANALISIS.md
│   └── erd.png
├── soal-2-trigger/
│   ├── 01_init_tables.sql
│   ├── 02_trigger_function.sql
│   ├── 03_concurrency.sql
│   ├── 04_test_cases.sql
│   └── ANALISIS.md
├── soal-3-fullstack/
│   ├── frontend/
│   │   ├── index.html
│   │   ├── app.js
│   │   └── prompts/gemini_canvas_prompt.md
│   ├── backend/
│   │   ├── audit_trail.sql
```

```
│   │   └── reversing_entry_rpc.sql
│   ├── psak_verification.md
│   └── ANALISIS.md
└── docs/
    ├── LAPORAN_UTS.md
    ├── LAPORAN_UTS.pdf
    └── screenshots/
        ├── soal-1/
        ├── soal-2/
        └── soal-3/
```

B. Tools yang Digunakan

Tools	Fungsi
PostgreSQL / Supabase	Database & SQL sandbox
Gemini Canvas	Generate UI HTML + Tailwind
NotebookLM	Verifikasi kepatuhan PSAK 2
dbdiagram.io	Merancang ERD
VS Code + Git	Editor & version control
GitHub	Repositori publik

C. Link Repositori

GitHub Public:
https://github.com/arfarori23/UTS-Coding-Akuntansi-Arfa-12030124130110

Laporan PDF (direct link):
https://github.com/arfarori23/UTS-Coding-Akuntansi-Arfa-12030124130110/blob/main/docs/LAPORAN_UTS.pdf

D. Screenshot Bukti Uji

- `docs/screenshots/soal-1/` — Uji DDL & constraint (8 test)
- `docs/screenshots/soal-2/` — Uji trigger persediaan (7 test)
- `docs/screenshots/soal-3/` — Uji immutability, reversing entry, UI balance

E. AI Tools Log

- **Gemini Canvas:** prompt lengkap di `soal-3-fullstack/frontend/prompts/gemini_canvas_prompt.md`
- **NotebookLM:** hasil verifikasi di `soal-3-fullstack/psak_verification.md`

Laporan ini disusun sebagai bagian dari Ujian Tengah Semester Gasal 2026/2027
Departemen Akuntansi — Fakultas Ekonomika dan Bisnis — Universitas Diponegoro